Assignment 9: Cache Memory

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

August 17, 2026

Deadline: **Tuesday, 1 September 2026, 11:59 PM** — exactly *two weeks* from issue (18 August 2026). There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions.

9.1 Conceptual

Q1. The entire cache design rests on two kinds of *locality of reference*.

(a) Define **temporal locality** and **spatial locality**, giving one concrete example of each from a running program (not a paraphrase of the definitions).

(b) Explain which kind of locality justifies fetching a whole *block* of neighbouring bytes on a miss (rather than the single word asked for), and which kind justifies choosing a replacement policy that keeps recently-used blocks resident (rather than, say, evicting the first block that was loaded).

Q2.

(a) Explain why a *direct-mapped* cache needs *no replacement algorithm at all*, whereas a fully-associative cache and a k-way set-associative cache each need one. Where, exactly, does the replacement "decision" arise?

(b) Exact LRU over a 2-way set-associative cache needs a single USE bit per set, but exact LRU over a 64-line fully-associative cache needs to remember roughly a 64-way *ordering* of lines (about 296 bits). Explain in words where this cost blow-up comes from — what does "recency order" mean as a quantity, and how does its size grow with the number of lines?

(c) Optimal (OPT) replacement is *unimplementable* in a real cache. State why, and state what purpose it nonetheless serves in cache design.

9.2 Numerical

Q3. A cache system — call it *System B* — is specified as follows: main memory **64 KB**, cache **4 KB**, block (line) size **16 bytes**. Following the lecture's *derive-the-fields* method (never memorise), compute, for each of direct, fully-associative, and 4-way set-associative mapping:

(a) the physical address width N and the word (offset) field width b (these are shared by all three schemes);

(b) the number of cache lines L, and (for set-associative) the number of sets S;

(c) the width in bits of the tag and index fields;

(d) the total number of main-memory blocks, and how many blocks compete for each line/set.

For each scheme, write the address split as `tag` | `index` | `word` (or `tag` | `word` for fully associative) with the widths written in, and show the width check that sums to N.

Q4. In the same *System B* (64 KB memory, 4 KB cache, 16-byte blocks), trace the address `0x2F9C` through all three mapping schemes.

(a) Direct mapping: which line does `0x2F9C` map to, and what is its tag and word offset?

(b) 4-way set-associative mapping: which set does it map to, and what is its tag and word offset?

(c) Fully-associative mapping: what is its tag and word offset, and which line(s) may it occupy?

Show your working *both* ways: bit-slice the 16-bit address into fields, *and* re-derive the same values arithmetically from the block number ($B = \lfloor \text{addr}/16 \rfloor$, line/set $= B \bmod L$ or $B \bmod S$, tag $= \lfloor B/L \rfloor$ or $\lfloor B/S \rfloor$). The two routes must agree.

Q5. A *fully-associative* cache with **3 lines**, initially empty, receives the following stream of block numbers:

$$2,\ 3,\ 2,\ 1,\ 5,\ 2,\ 4,\ 5,\ 3,\ 2,\ 5,\ 2 \qquad (12 \text{ references}).$$

For each of **LRU**, **FIFO**, **LFU**, and **OPT**, show the full trace — the contents of the three lines after every reference, marked `M/H` — and report the number of hits, the number of misses, and the hit ratio. For LFU, break ties in reference count by evicting the block loaded earliest. Rank the four policies by hit ratio.

Q6. (Belady's anomaly.) Consider the reference string

$$1,\ 2,\ 3,\ 4,\ 1,\ 2,\ 5,\ 1,\ 2,\ 3,\ 4,\ 5$$

under FIFO replacement on a fully-associative cache.

(a) Trace FIFO with **3** lines and count the hits and misses.

(b) Trace FIFO with **4** lines and count the hits and misses.

(c) State whether Belady's anomaly occurred, and in one sentence explain why the result is counter-intuitive.

(d) Now trace LRU on the same string with **3** lines and then with **4** lines. Does LRU show the same anomaly? Explain in words why LRU and OPT are "stack algorithms" and therefore immune, while FIFO is not.

Q7.

(a) A program makes 2500 memory references, of which 2350 hit in a cache whose access time is 5 ns. A miss costs a *further* 60 ns to fetch the block from main memory. Compute the hit ratio, the miss ratio, and the AMAT (average memory access time). State the sanity check that your AMAT answer must pass.

(b) A two-level hierarchy has L1 access time $t_1 = 2$ ns with hit ratio $h_1 = 0.90$; L2 access time $t_2 = 20$ ns with *local* hit ratio $h_2 = 0.80$ (measured over the accesses that miss in L1); and main-memory time $t_m = 120$ ns. Compute the effective access time under (i) *hierarchical* (sequential) access and (ii) *simultaneous* (parallel) access. State the *global* L2 hit ratio, and verify that the three probability weights in the hierarchical formula sum to 1.

9.3 Design

Q8. A cache has **128 lines** of **64-byte** blocks and is addressed by a **24-bit** address. Ignoring valid and dirty bits, complete the following table of hardware cost for direct, 2-way, 4-way, 8-way, and fully-associative organisations:

Organisation	**Sets**	**Index bits**	**Tag bits**	**Comparators**	**Tag storage**
Direct (k=1)					
2-way					
4-way					
8-way					
Fully associative					

Using the trade-off this table exposes (comparator count and tag storage rise as associativity rises, while conflict misses fall), recommend and justify an associativity for (i) a one-cycle L1 cache and (ii) a twenty-cycle L3 cache.

Q9. A processor runs a write-heavy workload (the same block is written many times in a row), and the system also contains a DMA controller that moves blocks between main memory and an I/O device.

(a) Define **write-through** and **write-back**. For the write-heavy workload, which generates less traffic to main memory, and why?

(b) Describe, step by step, what a write-back cache does when (i) the CPU writes to a block that is already resident (a write *hit*), and (ii) a *dirty* line is later evicted.

(c) The DMA controller reads blocks directly from main memory. Explain why this is a correctness problem for a write-back cache, and what must happen before the DMA transfer so that the device observes up-to-date data.